그래프DB 구축: 검색 저장소 구성 4가지 안 비교

벡터, 키워드, 그래프를 어디에 두느냐로 갈림 — 한국어 키워드 품질과 운영 부담이 선택을 가름

	1-a 분리형(저장소 따로)			1-b 분리형(검색 엔진)		2 벡터 통합형		3 전부 통합형		
	벡터 DB	키워드 색인	Neo4j	검색 엔진(nori)	Neo4j	Neo4j	키워드 색인	Neo4j		
	벡터	키워드	그래프	벡터 + 키워드	그래프	벡터 + 그래프	키워드	벡터	키워드	그래프
한국어 키워드 품질	좋음 형태소 분석기를 골라 씀(예: Kiwi)			좋음 nori 공식 플러그인과 사용자 사전		좋음 형태소 분석기를 골라 씀		주의 한국어 분석기 없음, 미리 나누거나 직접 개발		
운영: 저장소와 장애	보통 3곳 운영, 한 곳 멈춰도 나머지 검색			좋음 2곳, 검색 엔진 복제로 장애 대비		보통 2곳, Neo4j 멈추면 벡터와 그래프 함께 멈춤		주의 1곳으로 단순, 멈추면 검색 전부 멈춤		
운영: 규모와 백업	보통 저장소마다 확장과 백업을 따로 함			좋음 대규모, 필터, 백업 기능 갖춤		보통 Community는 온라인 백업과 클러스터 없음		주의 같은 제약에 키워드 색인까지 몰림		
운영: 증분 색인	주의 **지원: 벡터와 그래프** 키워드는 도구 따라 다름 3곳에 따로 쓰고 맞춤			보통 **지원: 모두** 2곳, 검색 엔진은 벡터와 키워드를 한 문서로 갱신		보통 **지원: 벡터와 그래프** 키워드는 도구 따라 다름 2곳, Neo4j와 키워드 맞춤		좋음 **지원: 모두** 1곳, 한 트랜잭션에서 벡터와 키워드 함께 갱신		
그래프 연결	보통 저장소 사이를 ID로 이어 조회			보통 같음(검색 결과 ID로 Neo4j 조회)		좋음 관계 선 하나, 쿼리 한 번		좋음 관계 선 하나, 쿼리 한 번		

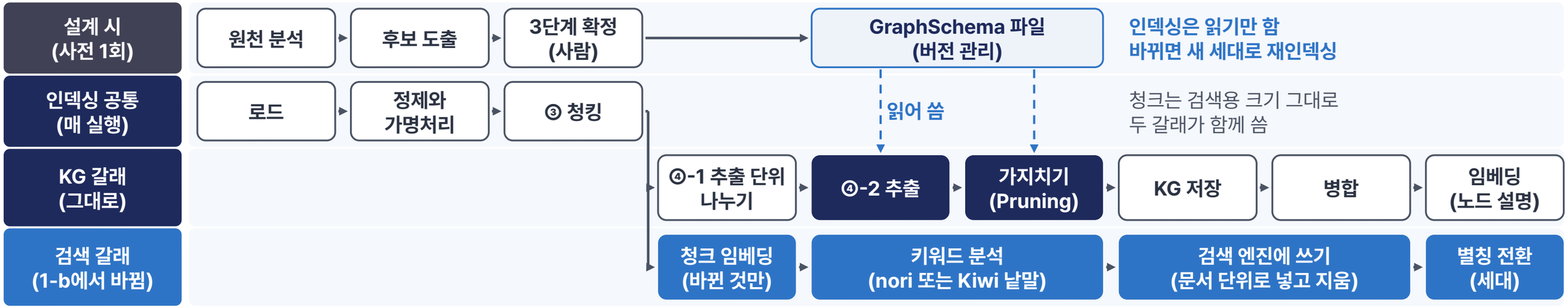
권고: 한국어 실서비스는 1-b 분리형(검색 엔진)이 정석, 그래프 비중이 크고 규모가 작으면 2 벡터 통합형
3 전부 통합형은 증분 색인이 가장 단순하지만 한국어 분석기와 필터를 Neo4j에서 직접 만들어야 해 한국어에서는 마지막 선택지

※ 키워드 색인을 추가와 삭제가 없는 정적 라이브러리(예: bm25s)로 만들면 바뀔 때마다 전체를 다시 만들어야 함(지금 우리 구성)

무단전재 및 배포 금지

그래프DB 구축: 1-b 분리형(검색 엔진)의 인덱싱 파이프라인

청킹까지 함께 쓰고 두 갈래로 나눔 — 검색 갈래는 검색 엔진에 문서 단위로 쓰고, KG 갈래는 지금과 같음



세대 전환을 함께

- 새 세대 인덱스를 다 만든 뒤 검색 엔진 별칭과 Neo4j 세대를 같은 순간에 바꿈
- 한쪽만 바뀌면 검색 결과의 chunk_id가 그래프에 없게 됨
- 증분 색인은 바뀐 문서만 두 곳에 쓰고 맞춤

검색 때 그래프로 건너감

- 검색 엔진이 하이브리드로 청크를 찾음(BM25와 벡터, 회원 필터)
- 돌려받은 chunk_id로 Neo4j 추출 단위 노드의 parent_chunk_ids를 찾음
- 관계를 따라 질문에 필요한 근거 노드를 얻음

검색 엔진 라이선스(2026-10 기준, 둘 다 nori 지원)

Elasticsearch	AGPL v3, SSPL, ELv2 중 선택 2024-08 AGPL 추가로 다시 오픈소스 AGPL로 고쳐서 서비스하면 소스 공개
OpenSearch	Apache 2.0(7.10.2에서 갈라짐) 2024-09부터 리눅스 재단 산하 재단 관리 고쳐 써도 소스 공개 의무 없음

추출 단위
LLM에 한 번에 넣는 글 조각



무단전재 및 배포 금지